

Chapter 1.1

Introduction to Mobile Application Development

Objectives

- Mobile operating systems and ecosystem
- Development tools
- Mobile devices
- Mobile application software architecture and framework

Mobile Operating Systems And Ecosystem

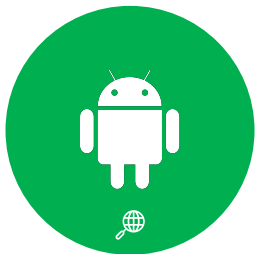
Mobile Operating Systems

Question?

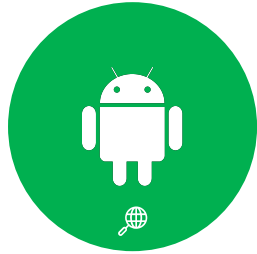
What is your phone OS?



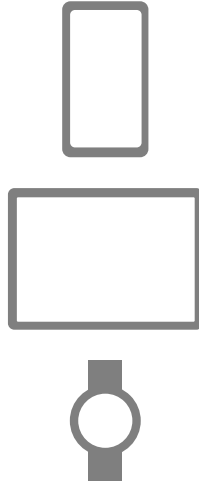
Mobile OS



Mobile OS - Android



Linux kernel
Open Source

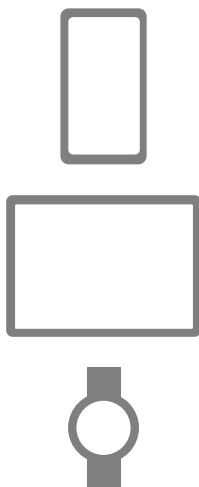


Chrome OS
Linux-based
Open Source

Mobile OS - iOS



Unix kernel



macOS

Mobile OS - Tizen



Linux kernel

Open Source



Mobile OS - KaiOS



Linux kernel

Open Source



Mobile OS – Harmony OS



Microkernel

Open Source



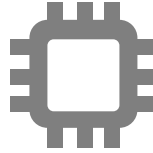
Mobile OS - Fuchsia



Microkernel

Open Source

Module Architecture



ARM-based



Nest Hub

Is Windows CE still available?

Windows CE Mobile Devices



How many types of mobile apps are there?

Types of Mobile Apps

Native



Hybrid

Web
+
Native

Web



Comparison of Apps

Criteria	Native	Hybrid	Mobile Web
Development cost	High	Low	Low
App performance	Fast	Depending on speed of network	Depending on speed of network
Distribution method	App stores	App stores	None
Access to device features	Wide	Limited	Very limited
Code maintenance	Multiple codebase	Single codebase	Single codebase

Question?

In your opinion, which mobile app development method (native, mobile web, and hybrid) is suitable for each the following entities?

- a. Bernama, the Malaysian national news-agency
- b. Super Mario Bros, a game developed by Sega
- c. Setapak Central, a shopping mall



Development Tools

Programming Languages

Native



Hybrid

Web
+
Native

Web



Development Tools - Native



Android
Studio



XCode



Tizen
Studio

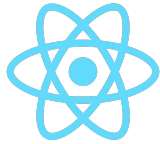


Visual
Studio



Huawei Quick
App

Development Tools – Cross Platform



React Native



Flutter



Xamarin



PhoneGap



Apache Weex



Native Script

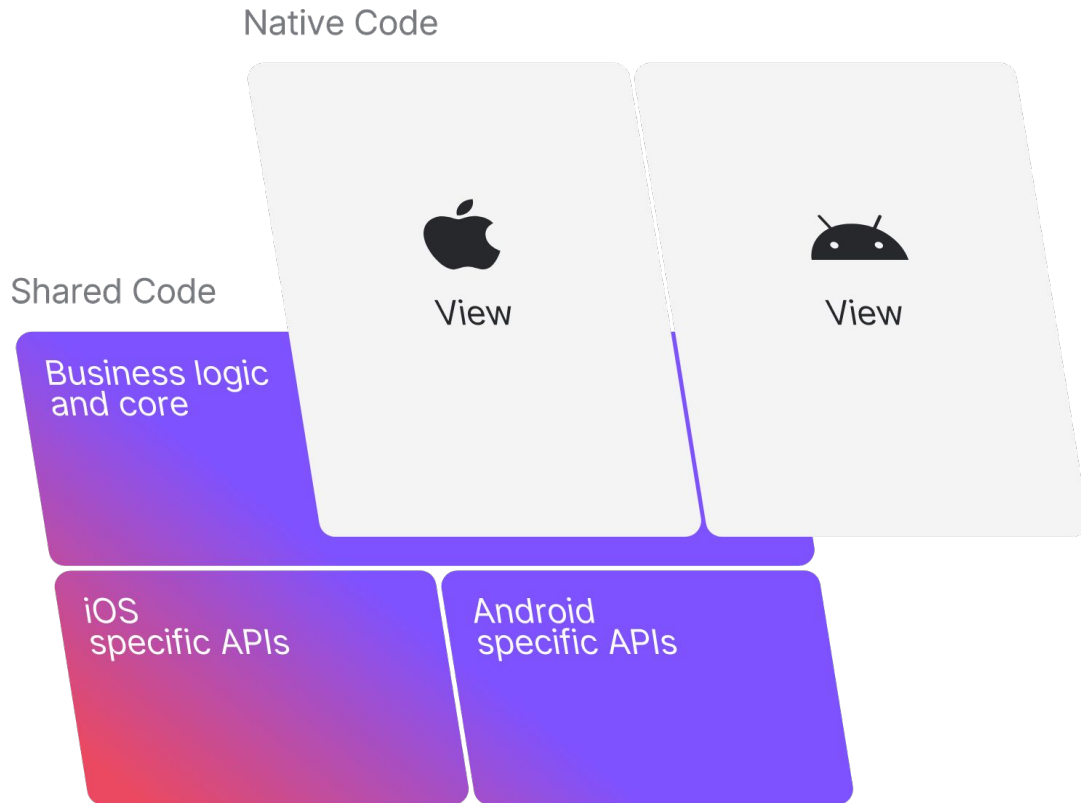


Jasonette



Ionic

Kotlin Multiplatform Mobile (KMM)



What are the apps developed using KMM?

Real-world success stories



Mobile Devices

Mobile devices



Flagship



Mid Range



Entry



Basic



4610 MHz



1300 MHz



24++ GB RAM / 1TB
ROM



4 GB RAM / 64 GB
ROM



6000 mAh



2150 mAh

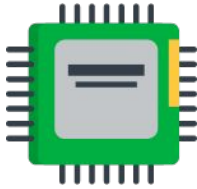


5G Network

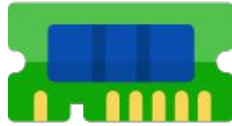


4G Network

Key Mobile Challenges



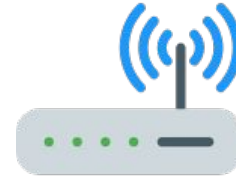
Processing
Power



Memory and
storage



Battery



Network

Beyond Mobile Devices

Beyond Mobile App



Wearable



Smart Home Appliance



Smart Car Dashboard



Internet of Things
(IOT)

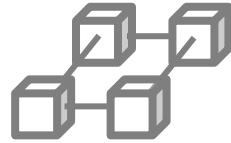
Beyond Mobile App



Augmented Reality
(AR)



Artificial Intelligent
(AI)



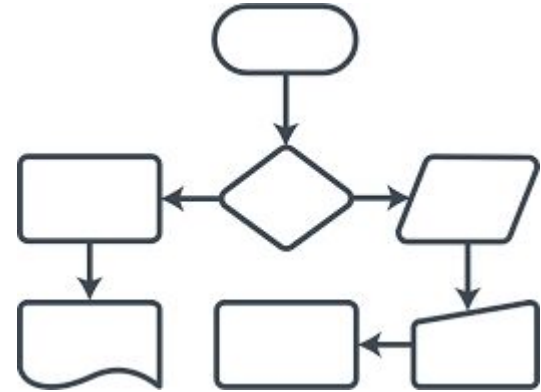
Blockchain

AI Native

AI Native

The focus has shifted from static interfaces to **dynamic, context-aware experiences**.

In the past, developers built fixed paths (screens and buttons); today, they build core logic that an AI agent interprets to fulfill user intent.



AI Native

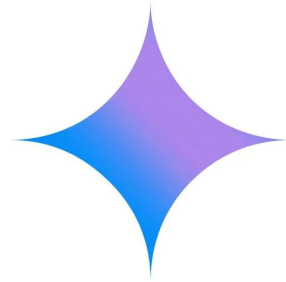
Vibe Coding & AI Collaboration

Tools like GitHub Copilot and specialized AI IDEs have turned coding into a high-level architectural task.

Developers often "vibe code" by describing functionality, allowing AI to handle boilerplate, API integration, and even complex logic flows.



GitHub
Copilot



Gemini in Android Studio

AI Native

Lower Barrier to Entry

Beginners are now launching sophisticated, profitable apps solo by leveraging generative AI for everything from backend infrastructure to UI assets.



Mobile Application Software Architecture And Framework

Introduction to Flutter

An open-source mobile application development framework created by Google.

It is used to develop applications for Android and iOS, web and desktop (Windows, Mac OS, Linux, etc).

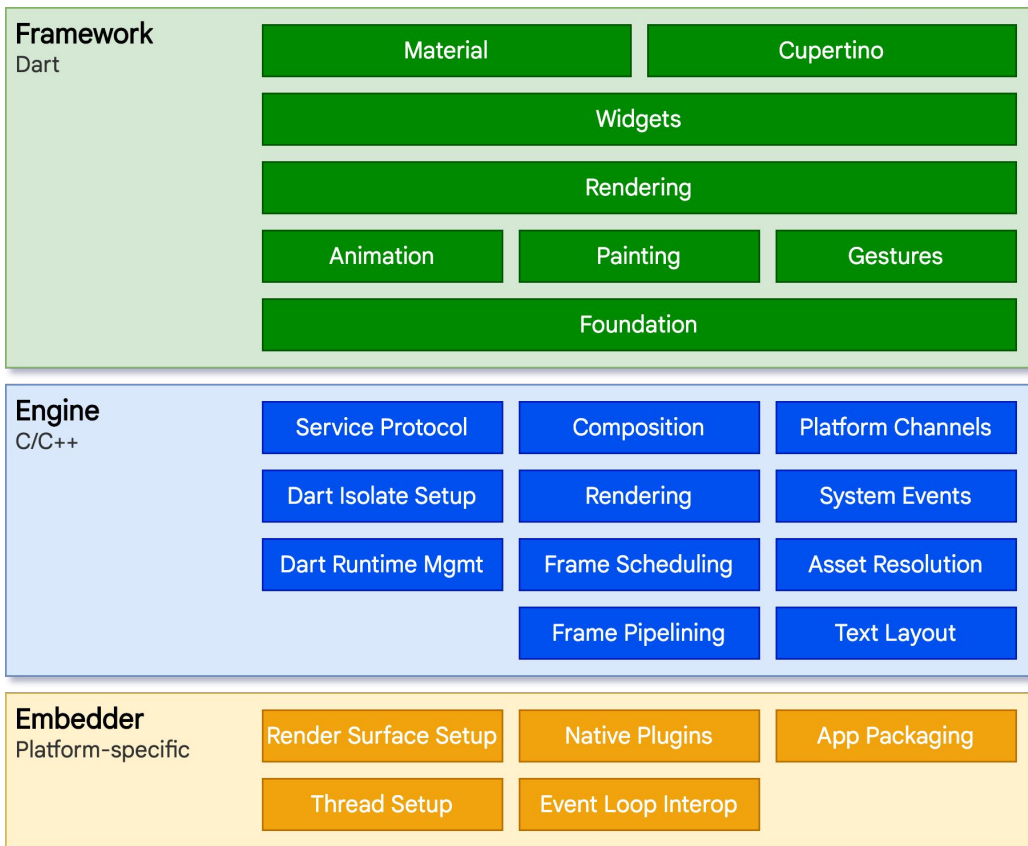
It is an SDK.

Language : DART

Key Features

- Cross-platform development
- Hot reload
- Rich widgets
- Customizable
- High performance
- Open-source

Architectural

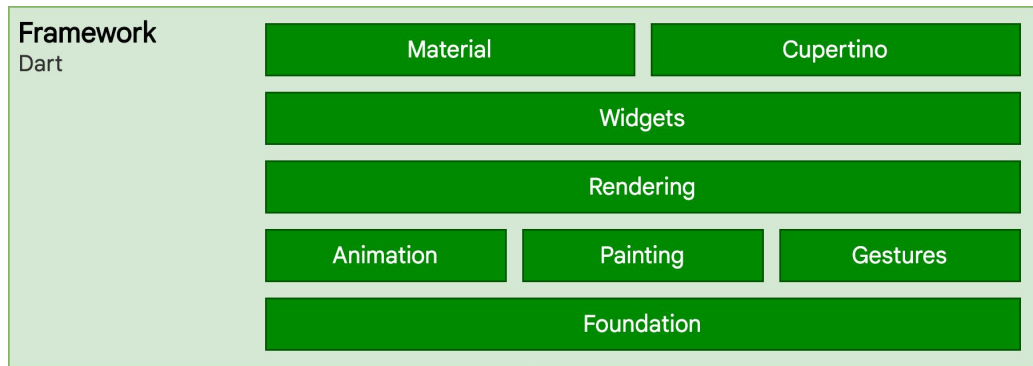


Architectural - Framework

A programming language used to build Flutter apps.

Offers features like strong typing, asynchronous programming, and just-in-time (JIT) compilation for rapid development.

Ahead-of-time (AOT) compilation for production builds to improve performance.

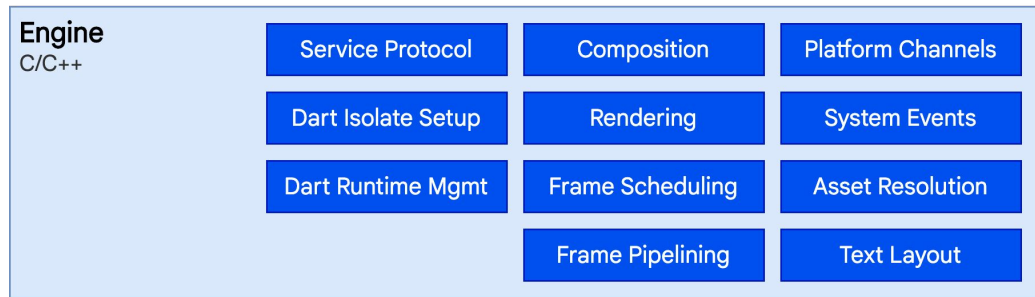


Architectural - Engine

The foundation of Flutter, written mainly in C++.

Handles platform-specific tasks such as rendering, input, and network requests.

Implements the Flutter framework and Dart runtime.



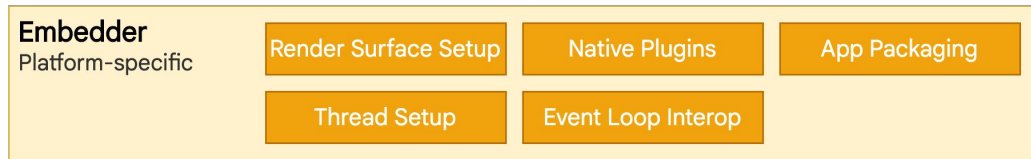
Architectural - Embedder

A platform-specific embedder coordinates with the underlying operating system.

Written in a language that is appropriate for the platform:

- Java and C++ for Android
- Objective-C/Objective-C++ for iOS and macOS
- C++ for Windows and Linux

With embedder, Flutter code can be integrated into an existing application as a module, or the code might be the entire content of the application.



Architectural - Reactive UI

It focuses on data flow and propagation of change.

Building UI that react to changes in data in a declarative and efficient manner.

Three key concepts:

1. Data Flow
2. Declarative UI
3. State Management

Architectural - Data Flow

1. Data Flow

Data flows unidirectionally from the data source to the UI. Changes to the data source trigger updates to the UI.



Architectural - Data Flow

1. Data Flow

Reactive UI - a programming paradigm that focuses on data flow and propagation of change.

UI automatically update whenever the underlying data changes.

This leads to more responsive, efficient, and maintainable applications; one-way flow makes it easier to track and debug state changes, as you always know where the data is coming from.

Architectural - Reactive UI

2. Declarative UI

You describe the desired UI state, and Flutter handles the rendering and updates.

This makes it easier to reason about the UI and write less code.

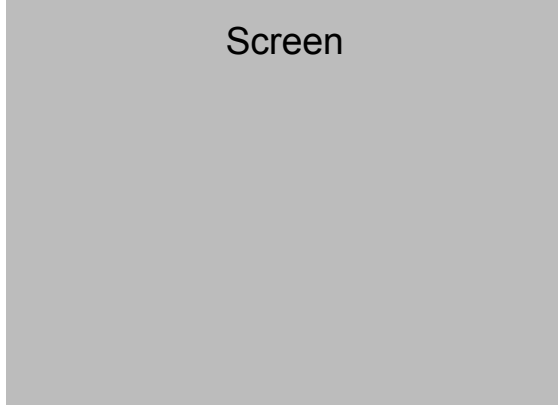
Widget is the fundamental of UI.

```
//Examples of Text widget
Text( 'You have pushed the button this many times:', ),
Text( '$_counter', style: Theme.of(context).textTheme.headline4, ),
```

Architectural - Reactive UI

2. Declarative UI

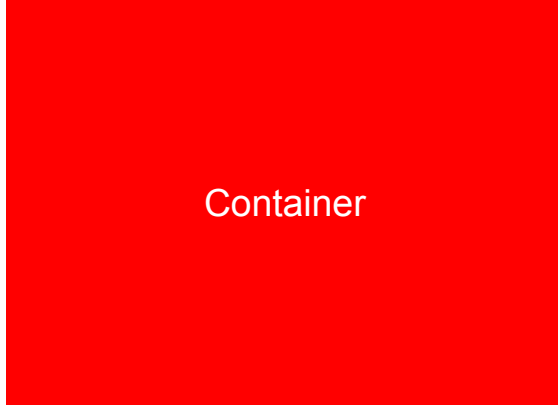
The screen is the parent (the root) of all widgets.



Architectural - Reactive UI

2. Declarative UI

E.g.1: The Container is forced to be exactly the same size as the screen.

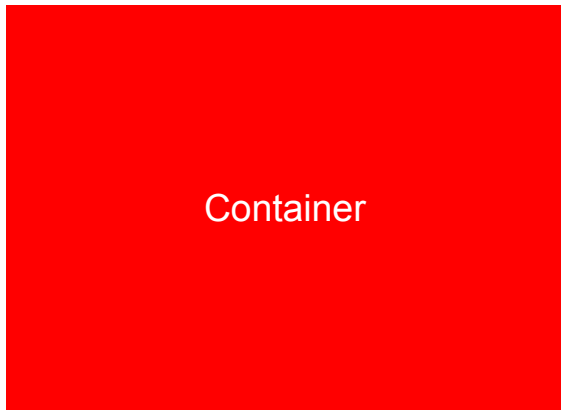


```
Container(color: red)
```

Architectural - Reactive UI

2. Declarative UI

E.g.2: The red Container wants to be 100×100 , but it can't, because the screen forces it to be exactly the same size as the screen.

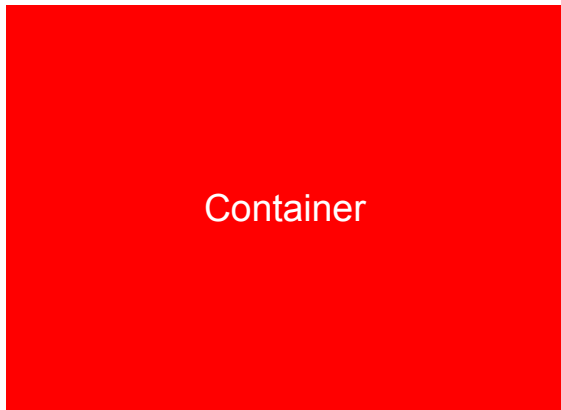


```
Container(  
  width: 100,  
  height: 100,  
  color: red)
```

Architectural - Reactive UI

2. Declarative UI

E.g.3: The screen forces the Center to be exactly the same size as the screen, so the Center fills the screen.

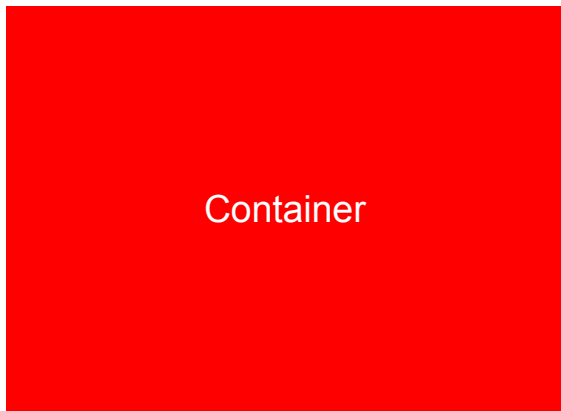


```
Center(  
  child: Container(color: red),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.4: The Center tells the Container that it can be any size but not bigger than the screen. The Container wants to be of infinite size, so it just fills the screen.

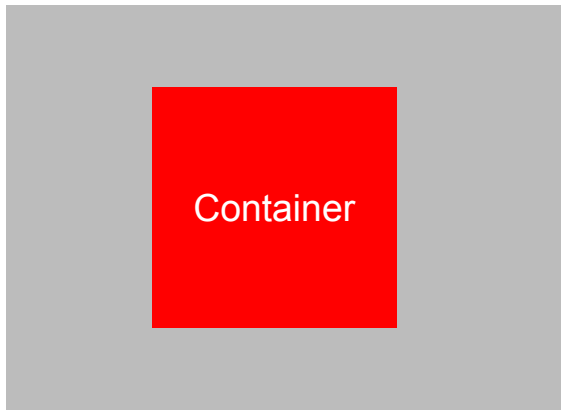


```
Center(  
  child: Container(  
    width: double.infinity,  
    height: double.infinity,  
    color: red),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.5: The Center is the same size as the screen. The Center ensures that the Container is not bigger than the screen. So, the Container can be 100 × 100.

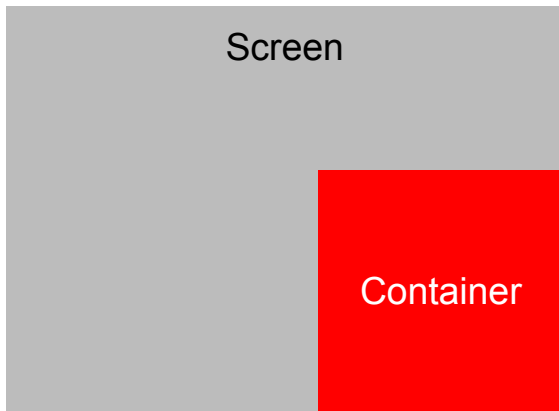


```
Center(  
  child: Container(  
    width: 100,  
    height: 100,  
    color: red),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.6: Align tells the Container that it can be any size. It aligns the Container to the bottom-right of the available space.

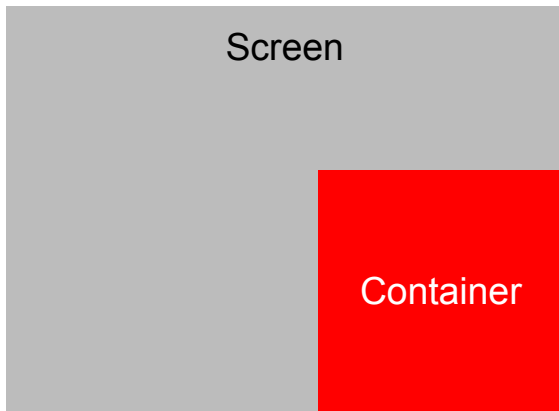


```
Align(  
  alignment: Alignment.bottomRight,  
  child: Container(  
    width: 100,  
    height: 100,  
    color: red),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.6: Align tells the Container that it can be any size. It aligns the Container to the bottom-right of the available space.

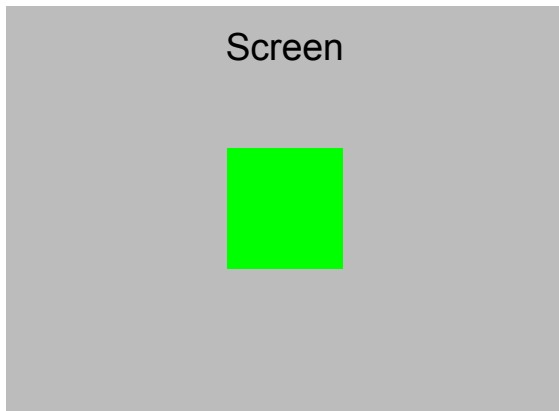


```
Align(  
  alignment: Alignment.bottomRight,  
  child: Container(  
    width: 100,  
    height: 100,  
    color: red),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.7: The Center tells the red Container that it can as big as the screen.
However, the red Container has no size, it decides it wants to be the same size as its child.

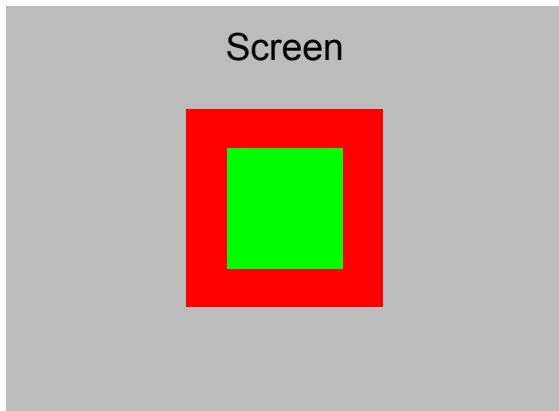


```
Center(  
  child: Container(  
    color: red,  
    child: Container(  
      color: green,  
      width: 30,  
      height: 30),  
    ),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.8: The red Container sizes itself to its children's size, but it takes its own padding into consideration. So it is also 30×30 plus padding.

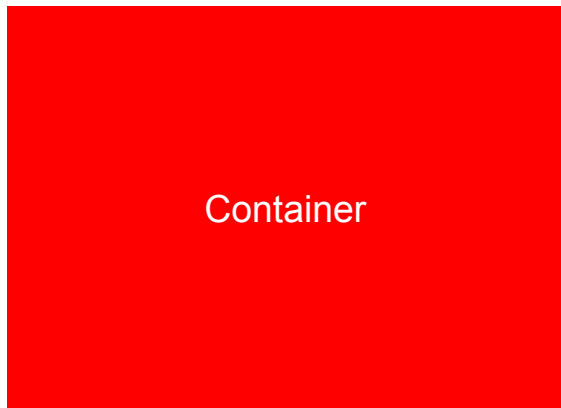


```
Center(  
  child: Container(  
    padding: const EdgeInsets.all(20),  
    color: red,  
    child: Container(  
      color: green,  
      width: 30,  
      height: 30  
    ),  
  ),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.9: The `ConstrainedBox` is forced to be exactly the same size as the screen, so it tells its child `Container` to also assume the size of the screen, thus ignoring its `constraints` parameter.

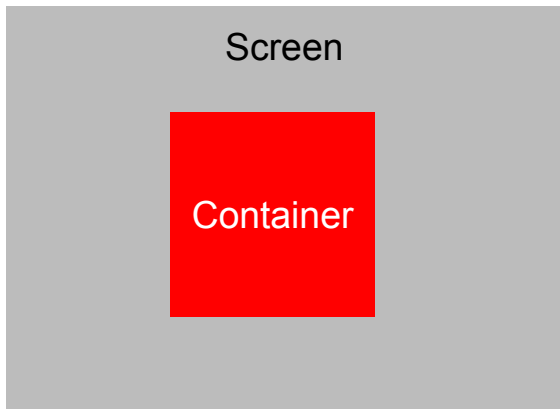


```
ConstrainedBox(  
  constraints: const BoxConstraints(  
    minWidth: 70,  
    minHeight: 70,  
    maxWidth: 150,  
    maxHeight: 150,  
  ),  
  child: Container(color: red, width: 10,  
                   height: 10),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.10: The Container must be between 70 and 150 pixels. It wants to have 10 pixels, so it ends up having 70 (the minimum).

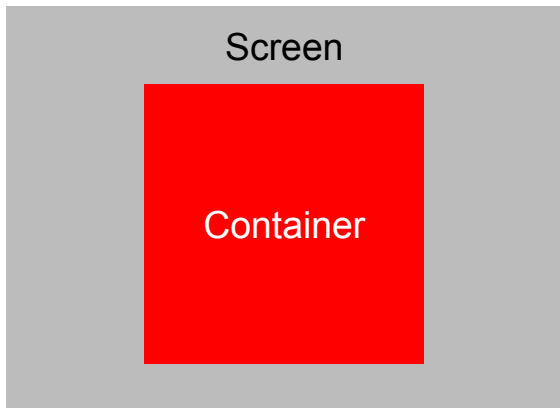


```
Center(  
  child: ConstrainedBox(  
    constraints: const BoxConstraints(  
      minWidth: 70, minHeight: 70,  
      maxWidth: 150, maxHeight: 150,  
    ),  
    child: Container(color: red, width: 10,  
                     height: 10),  
  ),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.11: The Container must be between 70 and 150 pixels. It wants to have 1000 pixels, so it ends up having 150 (the maximum).

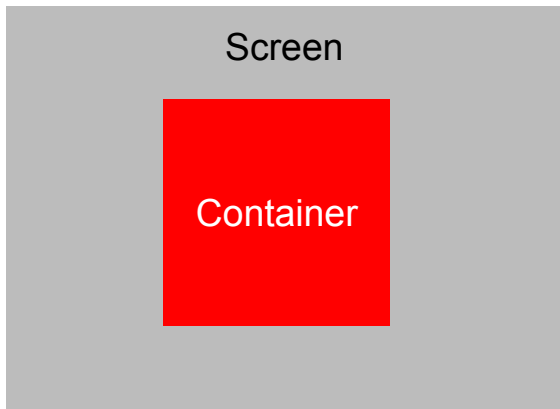


```
Center(  
  child: ConstrainedBox(  
    constraints: const BoxConstraints(  
      minWidth: 70, minHeight: 70,  
      maxWidth: 150, maxHeight: 150,  
    ),  
    child: Container(color: red, width: 1000,  
                     height: 1000),  
  ),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.12: The Container must be between 70 and 150 pixels. It wants to have 100 pixels, and that's the size it has, since that's between 70 and 150.

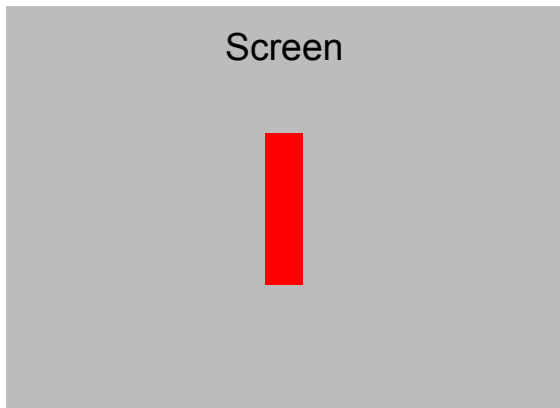


```
Center(  
  child: ConstrainedBox(  
    constraints: const BoxConstraints(  
      minWidth: 70, minHeight: 70,  
      maxWidth: 150, maxHeight: 150,  
    ),  
    child: Container(color: red, width: 100,  
                     height: 100),  
  ),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.13: The screen forces the `UnconstrainedBox` to be exactly the same size as the screen. However, the `UnconstrainedBox` lets its child `Container` be any size it wants.

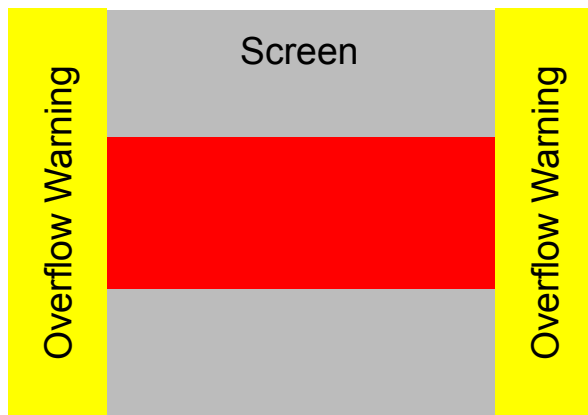


```
UnconstrainedBox(  
  child: Container(  
    color: red,  
    width: 20,  
    height: 50),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.14: The Container is 4000 pixels wide and is too big to fit in the UnconstrainedBox, so the UnconstrainedBox displays the much dreaded "overflow warning".



```
UnconstrainedBox(  
  child: Container(  
    color: red,  
    width: 4000,  
    height: 50),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.15: The Container has 4000 pixels of width, and is too big to fit in the OverflowBox, but the OverflowBox simply shows as much as it can, with no warnings given.

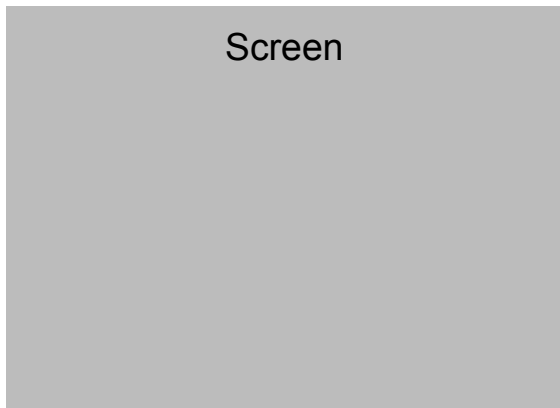


```
OverflowBox(  
  minWidth: 0,  
  minHeight: 0,  
  maxWidth: double.infinity,  
  maxHeight: double.infinity,  
  child: Container(color: red, width: 4000,  
                   height: 50),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.16: The system can't render infinite sizes, so it throws an error with the following message: BoxConstraints forces an infinite width.

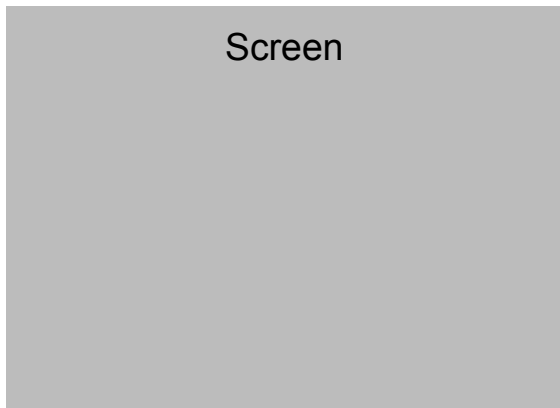


```
UnconstrainedBox(  
  child: Container(  
    color: Colors.red,  
    width: double.infinity,  
    height: 100),  
)
```

Architectural - Reactive UI

2. Declarative UI

E.g.17: The system can't render infinite sizes, so it throws an error with the following message: BoxConstraints forces an infinite width.



```
UnconstrainedBox(  
  child: Container(  
    color: Colors.red,  
    width: double.infinity,  
    height: 100),  
)
```

Architectural - Reactive UI

2. Declarative UI

“Constraints go down. Sizes go up. Parent sets position.”

- A widget can decide its own size only within the constraints given to it by its parent.
- Widget's parent who decides the position of the widget.
- It's impossible to precisely define the size and position of any widget without taking into consideration the parent-child tree as a whole.
- If a child wants a different size from its parent and the parent doesn't have enough information to align it, then the child's size might be ignored.

Architectural - State Management

State refers to the condition or data of a system at a particular moment.

It's a snapshot of the system's variables, properties, and other data that define its behavior.

In Flutter, state determines the dynamic behaviour of your app. Two main types of state:

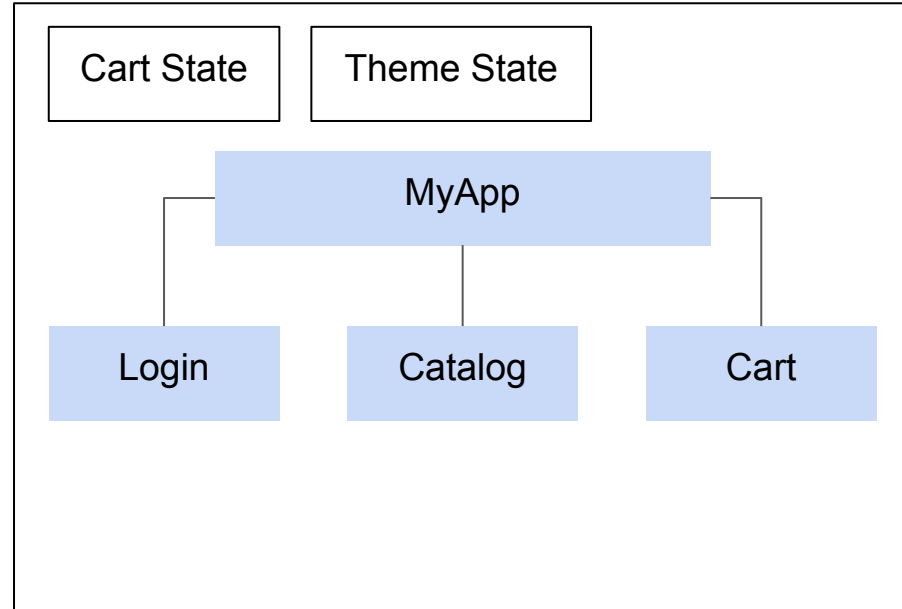
1. App State
2. Widget State

Architectural - State Management

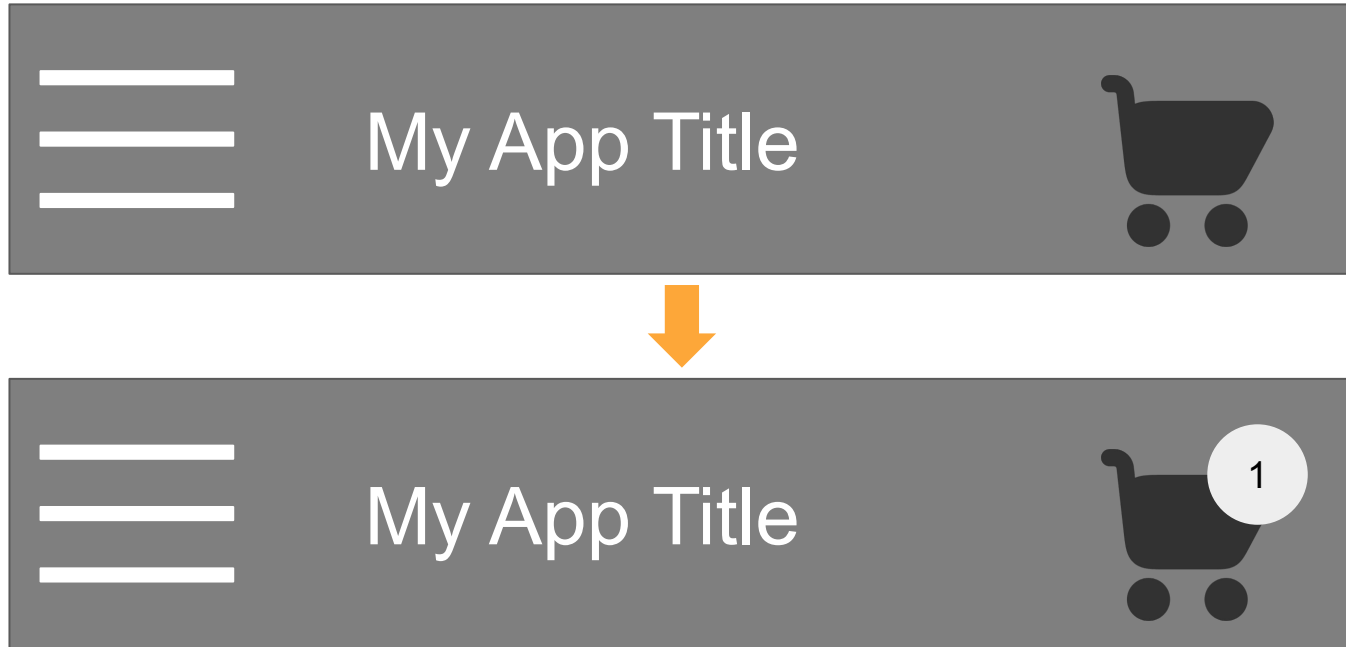
1. App State

Global state that affects the entire app.

Managed using libraries like Provider, Riverpod, or BLoC.



Architectural - State Management

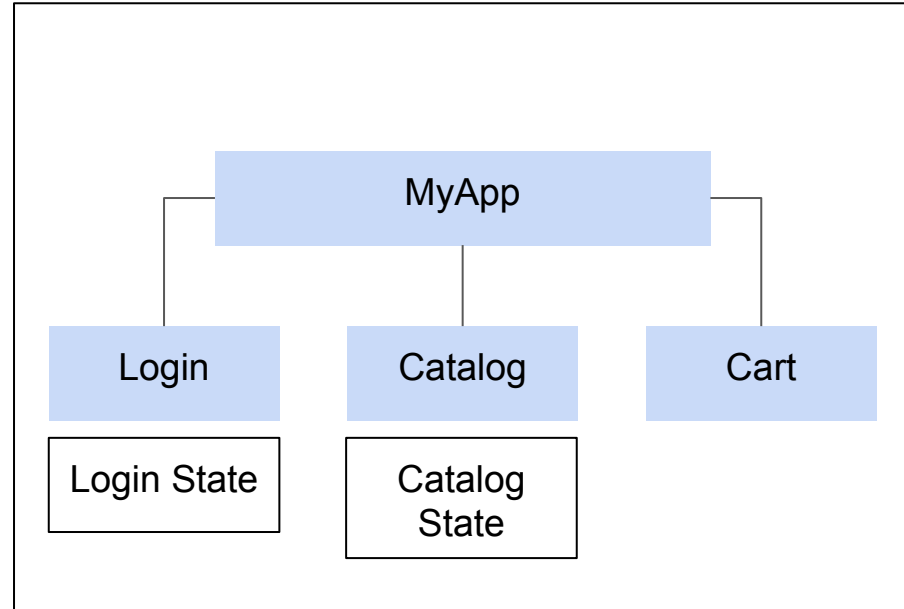


Architectural - State Management

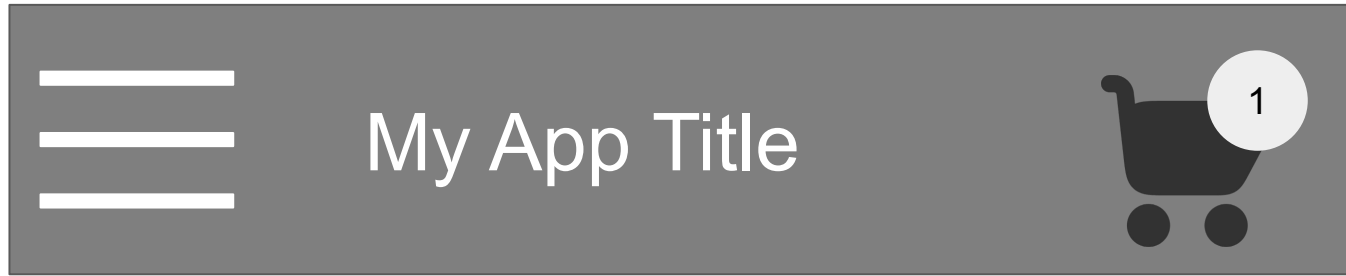
2. Widget State

Local state specific to a particular widget.

Managed using `StatefulWidget` and `setState`



Architectural - State Management



Item 1

Item 2



Item 3

React Framework

Flutter is a reactive, declaration UI framework.

Widgets are the building blocks of a Flutter app's UI.

Developer provides:

1. A mapping from application state to UI.
2. The framework takes on the task of updating the UI at runtime when the application state changes.

React Framework

Feature	Traditional UI	Reactive UI
Programming Paradigm	Imperative	Declarative
State Management	Manual	Framework-assisted
UI Updates	Manual	Automatic
Performance	Can be less efficient	More efficient

<pre>class MainActivity : AppCompatActivity() { private lateinit var textView: TextView</pre>	Declare UI state
<pre> override fun onCreate(savedInstanceState: Bundle?) { super.onCreate(savedInstanceState) setContentView(R.layout.activity_main)</pre>	Layout of UI
<pre> textView = findViewById(R.id.textView)</pre>	Linking UI to state
<pre> // Button click handler button.setOnClickListener { val newText = "Hello, World!" textView.text = newText } } }</pre>	Update state and UI

```
class MyHomePageState extends State<MyHomePage> {
```

```
  String _text = "";
```

Declare UI state

```
  void _toggleText() {
```

```
    setState(() { _text = 'Hello, World!'});
```

Update state

```
  }
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Scaffold(
```

```
    appBar: AppBar(
```

```
      title: const Text('My App'),
```

```
    ),
```

```
    body: Center(
```

```
      child: Column(
```

```
        mainAxisAlignment: MainAxisAlignment.center,
```

```
        children: [
```

```
          Text( _text, style: const TextStyle(fontSize: 24),
```

```
        ),
```

```
          ElevatedButton( onPressed: _toggleText,
```

```
            child: const Text('Click Me'),
```

```
        ...
```

Layout of UI

```
}
```

React Framework - Widget

Widgets form a hierarchy based on composition.

Each widget nests inside its parent and can receive context from the parent.

This structure carries all the way up to the root widget (the container that hosts the Flutter app, typically `MaterialApp` or `CupertinoApp`).

Note: We will discuss widgets in more detail...

Conclusion

- Mobile operating systems and ecosystem
- Development tools
- Mobile devices
- Mobile application software architecture and framework

Find Out More

Flutter architectural overview,

<https://docs.flutter.dev/resources/architectural-overview>